CMPSC 448 — Midterm Project

Student: Yishuo Shi

1. Introduction

Large Language Models (LLMs), such as ChatGPT, Claude, and DeepSeek, are widely used to generate text. Although they can answer the same questions, their writing styles and word choices may be different.

The goal of this project is to identify which LLM generated a response. Two neural network models, CNN and LSTM, are used to classify responses from three LLMs. This project focuses on two research questions:

RQ1: Can we identify which LLM generated a response?
RQ2: Does the user's input prompt help improve classification accuracy?

For RQ1, only LLM outputs are used. For RQ2, three input settings are compared: Input Only, Output Only, and Input + Output.

2. Dataset

2.1 Data Collection

The 500 questions were generated by Gemini based on several general topic directions. Gemini was only used for question generation and was not included in the classification task.

The same 500 questions were then answered by ChatGPT, Claude, and DeepSeek. Each model generated 500 responses, resulting in a dataset of 1,500 responses.

The dataset contains four columns: Question_ID, LLM_name, LLM_Input, and LLM_output. The LLM_name column is used as the classification label.

2.2 Data Preprocessing

The dataset was checked for missing values, and no missing data was found. The LLM names were converted into numerical labels: ChatGPT (0), Claude (1), and DeepSeek (2).

All text was converted to lowercase and split into words. A vocabulary was built using the training data, keeping words that appeared at least twice. The final vocabulary contains 7,428 tokens.

Each response was converted into a sequence of 200 token IDs. Longer responses were truncated, while shorter ones were padded with zeros.

2.3 Data Splitting

The dataset was divided into 70% training, 15% validation, and 15% testing.

Dataset	Samples
Training	1050
Validation	225
Testing	225

The data was split based on Question_ID to prevent the same question from appearing in different sets.

3. Methodology

3.1 CNN Model

The CNN model contains an embedding layer, a convolutional layer, a max pooling layer, and a fully connected layer.

The embedding layer converts each token into a 128-dimensional vector. The convolutional layer uses 128 filters with a kernel size of 3. ReLU and max pooling are used to extract important text features.

The original CNN did not include dropout. A dropout layer with a rate of 0.3 was added later to reduce overfitting. The final fully connected layer outputs three classification scores.

3.2 LSTM Model

The LSTM model is used to compare its performance with CNN. It contains an embedding layer, an LSTM layer, a dropout layer, and a fully connected layer.

The embedding size and hidden size are both set to 128. The LSTM processes text sequentially to learn relationships between words. Padding tokens are excluded from the LSTM sequence processing.

A dropout rate of 0.3 is applied before the final classification layer, which outputs three scores.

3.3 Training Settings

Both models are trained using PyTorch. The Adam optimizer is used with a learning rate of 0.001, and Cross-Entropy Loss is used for classification.

The batch size is set to 32, and each model is trained for 10 epochs. Validation accuracy is calculated after each epoch, and the model with the highest validation accuracy is saved for testing.

The experiments are performed using CUDA GPU acceleration.

4. Experimental Results

4.1 RQ1: Output Only

For RQ1, CNN and LSTM were trained using only the LLM outputs. Both models used the same training, validation, and testing datasets.

Model	Test Accuracy	Macro F1
CNN	74.67%	0.7411
LSTM	56.89%	0.5743

The CNN achieved higher accuracy and F1 score than the LSTM. The accuracy difference was 17.78 percentage points.

This suggests that CNN was more effective at identifying LLM responses in this experiment. The LSTM had lower performance, although its training loss decreased during training.

4.2 CNN Improvement

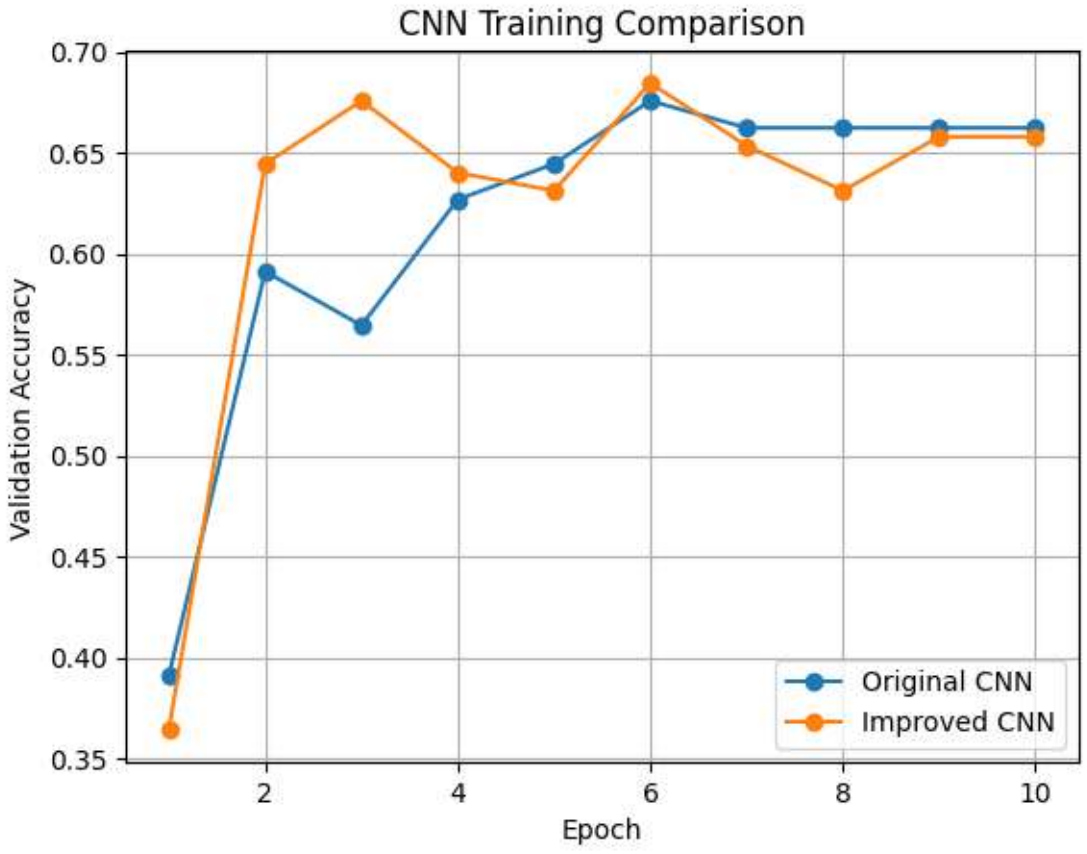


Figure 1. Validation accuracy of the original CNN and CNN with dropout.

The original CNN achieved 70.22% test accuracy and a Macro F1 score of 0.7031.

During training, the training loss continued to decrease, but the validation accuracy

stopped improving. This suggested possible overfitting.

To address this issue, a dropout layer with a rate of 0.3 was added. The training process was also modified to save the model with the highest validation accuracy.

The improved CNN achieved 74.67% test accuracy and a Macro F1 score of 0.7411.

The improved model performed better in this experiment. However, the difference may also be affected by random initialization and the updated model selection method.

4.3 RQ2: Input Comparison

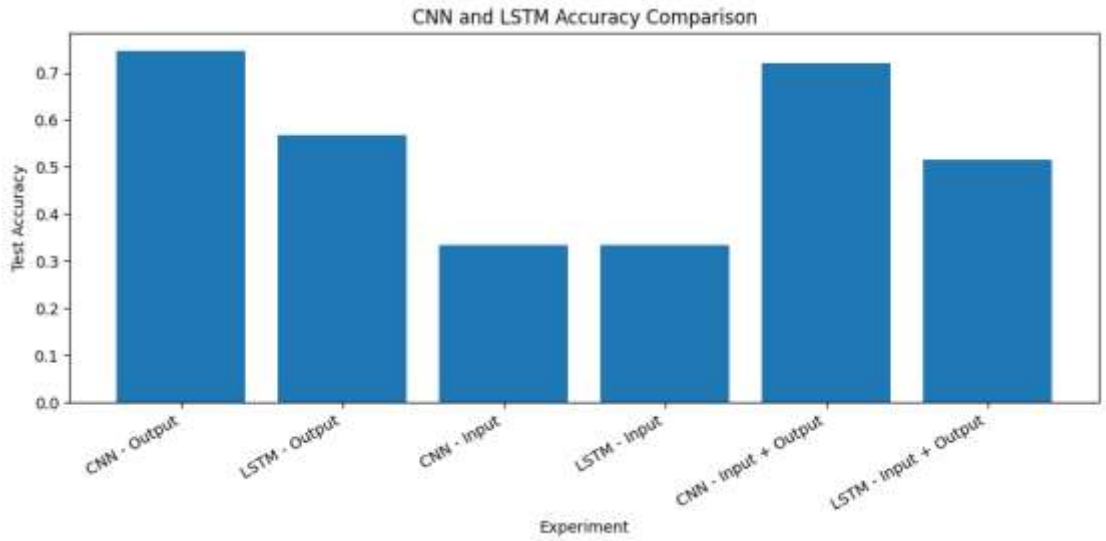


Figure 2. Test accuracy comparison of CNN and LSTM under three input settings.

For RQ2, three input settings were compared: Input Only, Output Only, and Input + Output.

Model	Input	Test Accuracy	Macro F1
CNN	Input Only	33.33%	0.2661
CNN	Output Only	74.67%	0.7411
CNN	Input + Output	67.11%	0.6739
LSTM	Input Only	33.33%	0.3296

LSTM	Output Only	56.89%	0.5743
LSTM	Input + Output	54.22%	0.5404

Both models achieved only 33.33% accuracy when using Input Only, which is equal to random guessing for three balanced classes.

Output Only achieved the highest accuracy for both models. Adding the input prompt did not improve performance in this experiment.

5. Discussion

The results show that CNN performed better than LSTM in all three input settings. For Output Only, CNN achieved 74.67% accuracy, while LSTM achieved 56.89%.

Both models achieved 33.33% accuracy for Input Only. This is because the same questions were given to all three LLMs. Each question has three different labels, so the input prompt alone cannot identify which model generated the response.

Input + Output also performed worse than Output Only. One possible reason is the sequence length limit of 200 tokens. When the input and output were combined, some parts of the responses may have been removed during truncation.

The results suggest that the LLM responses contain more useful classification information than the input prompts. CNN may be better at learning local writing patterns from this dataset.

6. Lessons and Experience

This project provided experience in text classification using CNN and LSTM models. It also helped improve understanding of text preprocessing, word embeddings, model training, and evaluation.

One challenge was overfitting in the CNN model. The training loss continued to decrease, but validation accuracy stopped improving. Adding dropout and saving the model with the highest validation accuracy helped improve the final test results.

Another important lesson was the effect of input selection. Input Only could not distinguish between the three LLMs, while Output Only achieved better results. This showed that choosing useful input features is important for classification.

The project also has some limitations. The dataset contains only 500 questions and three LLMs. The fixed sequence length of 200 tokens may also remove useful information. A larger dataset and different sequence lengths could be tested in future work.